A Dynamic Analysis-Powered Explanation Framework for Malware Detection

Huijuan Zhu, *Member, IEEE*, Xilong Chen, Liangmin Wang, *Member, IEEE*, Zhicheng Xu, and Victor S. Sheng, *Senior Member, IEEE*

***Abstract*—Deep learning has been widely adopted in Android malicious software (malware) detection. However, poor explanation in deep learning-based detection models severely undermines user trusts and poses a significant obstacle to their practical promotion in critical security domains. Some studies strive to uncover the rationale behind a model's decision. Unfortunately, these efforts are often hindered by the limitations of feature extraction methods, such as primarily relying on static analysis to derive separate and approximate behavioral descriptions of applications (apps). As a result, establishing a reliable interpretation for deep learning-based malware detection models remains an open issue. In this work, we propose a novel framework XDeepMal to interpret deep learning-based malware detection models. Specifically, in XDeepMal, we formulate a dynamic analysis tool XTracer$^+$ to capture runtime behaviors of apps and automatically generate their continuous behavior trajectories. Then, we propose a novel interpreter to pinpoint certainty behavior fragments that are crucial for deep learning models to make their decisions. This approach regards the identification of the most critical fragments as an optimization problem and leverages heuristic algorithms for implementation. We conduct extensive experiments on a real-world dataset to investigate the effectiveness and reliability of XDeepMal. These experiments cover intuitive case studies (malware family and individual app) and in-depth quantitative analysis. Additionally, we evaluate its coverage and efficiency. Our experimental results demonstrate that XDeepMal is capable of generating convincing interpretations for deep learning (e.g., Transformer) based models within feasible inference time, which greatly benefits security analysts in accurately comprehending why an app is identified as malware by deep learning-based detection models.**

***Index Terms*—Android, malware detection, explanation, dynamic behavior trajectory, transformer.**

Manuscript received 28 September 2023; revised 4 July 2024; accepted 29 July 2024. Date of publication 9 August 2024; date of current version 13 November 2024. This work was supported by the National Natural Science Foundation of China under Grant 62272204, Grant 62372105, and in part by the Leading-edge Technology Program of Jiangsu Natural Science Foundation under Grant BK20202001. Recommended for acceptance by X. Yi. *(Corresponding authors: Liangmin Wang; Victor S. Sheng.)*

Huijuan Zhu and Xilong Chen are with the School of Computer Science and Communication Engineering, Jiangsu University, Zhenjiang 212013, China (e-mail: huijuan-zhu@ujs.edu.cn; 2222108-058@stmail.ujs.edu.cn).

Liangmin Wang is with the School of Cyber Science and Engineering, Southeast University, Nanjing 211189, China (e-mail: liangmin@seu.edu.cn).

Zhicheng Xu is with the School of Mathematical Sciences, Jiangsu University, Zhenjiang 212013, China (e-mail: 3190113029@stmail.ujs.edu.cn).

Victor S. Sheng is with the Department of Computer Science, Texas Tech University, Lubbock, TX 79409 USA (e-mail: victor.sheng@ttu.edu).

This article has supplementary downloadable material available at https://doi.org/10.1109/TKDE.2024.3436891, provided by the authors.

Digital Object Identifier 10.1109/TKDE.2024.3436891

I. Introduction

THE openness of the Android platform and the prevalence of third-party app markets contribute to a low barrier of entry for application (app) development. However, this convenience is often accompanied by an insufficient security review process. As a result, there has been a persistent increase in the prevalence of Android malicious software (malware). Typical behaviors of Android malware, such as privacy theft, remote control, cost consumption, and malicious billing [1], are undermining the legitimate rights and interests of users [2]. To tackle these issues, deep learning has been increasingly employed for malware detection in recent years [3]. By automatically learning meaningful representations from abundant raw data, deep learning based methods have achieved impressive detection performance. For instance, based on the reviewed results in [1], the median number of accuracy/F1-score achieved by deep learning-based detection models is 98%. However, a major challenge posed by this kind of methods is their lack of interpretability, often referred to as black-boxes [4], raising trust-related concerns. This limitation significantly hampers their practical application and promotion.

In the malware detection community, sufficient and reliable interpretations for binary classification results (i.e., malicious and benign) yielded by deep learning models can help users or security experts understand why an app is identified as malware. Besides, it is an essential prerequisite for effectively tracking attack intent and formulating appropriate protective measures. For this reason, several explanation methods have been raised to probe the evidences for model decisions [5]. For instance, some more generic explanation methods, also known as model-agnostic, e.g., LIME [6] and SHAP [7], have been employed in the malware detection community. Unfortunately, the interpretations provided by different explanation approaches are often confusion [4]. There are also some pioneering explanation schemes proposed for specific deep learning-based models, which belong to model-specific ones. For instance, Wu et al. [8] proposed an explanation method by leveraging a customized attention mechanism with a multi-layer perceptron (MLP) model to provide the weights of separate features. By leveraging a Convolutional Neural Network (CNN) with an attention mechanism, Hiromu et al. [9] proposed a method that selectively highlights important malware family-specific regions within images converted from binary files, facilitating manual analysis of malware samples. Someya et al. [10] proposed an interpretable malware detection approach FCGAT, which utilizes static Function Call

1041-4347 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Jain University. Downloaded on January 05,2026 at 06:29:44 UTC from IEEE Xplore. Restrictions apply.

Graphs (FCG) and Graph Attention Network (GAT). Specifically, FCGAT provides an explanation component that ranks the importance of functions based on attention weights incorporated from a Long Short-Term Memory (LSTM) model.

These state-of-the-art methods significantly decrease the opacity in detection results obtained from deep learning models. However, existing explanation efforts primarily rely on coarse-grained features (e.g., existing or frequency) captured by static analysis methods. Some static analysis studies, such as FlowDroid [11] and MaMaDroid [12], extract sensitive information flows and package-level API sequences from a call graph to model the behaviors of apps. As static call graphs seek to approximate real program behaviors by illustrating all possible execution paths [13], it is uncertain which branches are actually executed in the real runtime environment. Meanwhile, static analyses are often infeasible, due to the intense use of dynamic features such as code obfuscation, dynamic code loading and reflection [14], [15], [16]. Therefore, these static features struggle to provide precise contextual semantics for explanation methods. The runtime behaviors of an app, defined by API call sequences and their contextual semantics, have proven effective in malware detection [15]. Despite its resilience to evasion techniques like obfuscation, dynamic analysis is used in only 17% of primary studies due to its higher computational cost [17]. Additionally, finding the 'sweet spot' to collect the minimum information necessary for accurate sample classification is challenging [18]. For these reasons, to the best of our knowledge, until now, there is no study that reliably interprets why an app is classified as malware by providing its precise runtime malicious behavior fragments.

In this work, to fill this gap, we propose a novel framework XDeepMal leveraging dynamic continuous behavior analysis to interpret deep learning-based malware detection models. Firstly, we formulate an automated dynamic analysis tool XTracer$^+$ to capture dynamic behavior trajectories for each app. The dynamic behavior trajectory in this work is defined as a sequence of Application Programming Interfaces (APIs)/permissions invoked in chronological order during app execution. Therefore, by considering the genuine behavior of an app, including its interactions with the system, we can establish a more reliable foundation for interpreting the detection results obtained from deep learning-based models. To effectively capture the contextual information within the captured behavior trajectories, we enhance our approach by employing a sequential model based on Transformer [19], [20]. This choice is motivated by the remarkable advantages of Transformer in effectively modeling long-term contextual correlations. Finally, we propose an explanation approach based on heuristic algorithms [21] to interpret the predicted results of our sequential model.

During the experimental evaluation phase, to avoid potential data quality bias in a single data source [22], we collect a large scale of apps from multiple sources (e.g., Google play store [23], VirusShare [24], Androzoo [25], [26], third-party app markets) to construct our dataset. Based on the constructed dataset, we conduct extensive experiments to investigate the effectiveness and reliability of XDeepMal, such as conducting a comprehensive comparison with similar explanation methods in terms of quantitative metrics, and then showcasing its performance through real-world case studies. We also evaluate its coverage and efficiency. Generally, our experimental results show that XDeepMal can pinpoint the suspicious malicious behavior fragments that drive the detection models to make their predictions, enabling users (e.g., security analysts) to exactly understand why an app is predicted as malware.

The main contributions of this work can be summarized as follows.

- We propose a novel explanation framework XDeepMal, which is the first framework focusing on why an app is classified as malware by providing its precise runtime malicious behavior fragments.
- In XDeepMal, we formulate a novel dynamic analysis tool XTracer$^+$ that enables the capture of the continuous runtime behavior trajectory exhibited by an app. We also propose a novel interpreter to pinpoint the crucial segments of the trajectory for a malware detected by a deep learning-based model.
- We conduct experiments to investigate the effectiveness and reliability of XDeepMal, including in-depth quantitative analysis compared with similar solutions, and intuitive case studies covering both the analysis of the malware family and individual samples. Our experimental results show that XDeepMal outperforms LIME by at least 15.49% in terms of average effectiveness, indicating that XDeepMal provides convincing interpretations for deep learning-based malware detection models to facilitate malware analysis.

The rest of the paper is organized as follows. Section II surveys related works. Section III outlines the architecture of the proposed method XDeepMal and presents a comprehensive overview of its components. Section IV describes our experimental results and conducts an analysis and discussion around these results. Finally, we conclude this work in Section V.

II. Related Work

In this section, we retrospect the existing learning-based malware detection solutions and summarize several explanation approaches for Android malware detection models. Finally, we illustrate our motivation.

A. Learning Based Malware Detection Methods

In recent years, machine learning (including deep learning) based methods have made significant progress in detecting Android malware [13]. These methods involve inputting the features extracted through static, dynamic, or hybrid analysis techniques to train classification models that can effectively identify malware [31]. In the context of static methodologies, Fairbanks et al. [32] employed a control flow graph as the primary feature and utilized the Random Forest algorithm for analysis. Sun et al. [33] focused on opcode features, employing a hybrid algorithm that combines CNN and Recurrent Neural Network (RNN) to identify malware. Chai et al. [31] proposed a dynamic prototype network that tackles the issue of detecting unknown malware. To achieve this, they converted executable files

Authorized licensed use limited to: Jain University. Downloaded on January 05,2026 at 06:29:44 UTC from IEEE Xplore. Restrictions apply.

into gray-scale images. In addition, high-level semantic features, such as API calls and permissions, are extensively employed in static analysis [17], [34], [35]. In the realm of dynamic analysis methods, Zhang et al. [36] extracted API calls and utilized a feature hashing technique to encode the API call arguments linked to their respective API names. And then a deep neural network architecture that combines CNN and LSTM is used for malware detection. Gong et al. [29] developed a dynamic analysis engine based on the official Google emulator, integrated with the Xposed hook framework. This engine enables the interception of target APIs before their actual invocation. They evaluated the performance of nine mainstream machine learning algorithms and found that Random Forest performs the best. Certainly, hybrid approaches that combine dynamic and static analysis techniques have emerged as effective solutions. For instance, Bai et al. [37] constructed a hybrid feature set for Random Forest (RF) to detect Android banking Trojan. Specifically, they extracted static features from the Android Manifest and the API Package and supplemented them with dynamic analysis using the Koodous online service. We notice that through static, dynamic and hybrid analysis, various types of features like APIs and permissions can be extracted for further analysis. Besides, control flow graphs, Opcode and API call graphs are also used to characterize apps [17].

The static analysis generally utilizes reverse engineering to obtain the crucial files (e.g., dex and Manifest files) of an Android application package (APK) file to further extract useful information. For instance, the requested permissions can be extracted from the Manifest file, and the API methods can be acquired from the dex files by static analysis tools (e.g., Androguard [38]). Dynamic analysis, on the other hand, involves monitoring the runtime behaviors (e.g., API calls, file system accesses, network accesses, etc.) of apps using technologies such as dynamic injection or hooking in sandbox. Dynamic analysis empowers analysts to precisely track the execution of an app at runtime, facilitating a deeper understanding of its behavior. Unfortunately, a comprehensive survey conducted by Liu et al. [1] presents that there are only 17% of existing efforts involve dynamic analysis. This is not surprising, as the high operational costs and the low success rate of app analysis considerably increase the difficulty of undertaking such work.

B. Explanation Methods for Malware Detection

Distinguished from improving the accuracy/precision of detection models, some studies [34], [39], [40] raise concerns that why these models predict an app as malware. The lack of transparency poses a considerable hurdle for security analysts to determine the trustworthiness of these decisions. For this reason, researchers have started working on enhancing the interpretability of deep learning-based models [41], [42]. For instance, in [32], the interpretability of detection models is realized by employing the Shapley Additive exPlanations (SHAP) technique. He et al. [13] designed an novel mechanism to explain the output of a Graph Neural Network (GNN) based classifier. It highlights questionable control flows and provides similar behavioral snippets of known malware to facilitate malware analysis. Iadarola et al. [39] converted the dex executable files of Apps to gray-scale images for CNN to detect malware, and used Gradient-weighted Class Activation Mapping (Grad-CAM) algorithm [43] and heatmap to provide insightful information to improve the interpretability of the detection model. These emerging explanation methods provide a new opportunity to unveil the decision-making mechanisms of black-box models, which is crucial for establishing user trusts in deep learning models.

C. Motivation

Although deep learning models have achieved impressive accuracy/precision in malware detection, the lack of explanation poses a major obstacle to their practical adoption [44]. Some explanation methods have been proposed for computer vision or other non-security domains, but they do not fully meet the specific requirements of the malware detection community. For instance, the rough localization within image explanation schemes has limited capabilities in precisely comprehending the specific behaviors of malware. Existing explanation efforts in malware detection are mostly based on static analysis, which makes it difficult for them to ensure the reliability of the interpretations derived from the feature source. For instance, the mere inclusion of certain permissions and/or APIs in the code does not guarantee that they will be actively executed or invoked at runtime to trigger attacks [16]. Besides, static analysis is vulnerable to widely used evasion techniques (e.g., reflection call, encryption) [14], [15], [16].

To acquire precision behaviors of apps, this work explores dynamic analysis. Typically, sensitive API calls can result in dangerous or malicious behaviors [1], which are widely concerned in the malware detection community. When utilizing these APIs, specific permissions are usually requested dynamically. Hence, the dynamic analysis in this work also focuses on API calls and permissions. Significantly, compared to the isolated consideration of API calls/permissions such as binary encoding or frequency statistics, we believe that continuous dynamic behaviors (dynamic behavior trajectories) can characterize the app more comprehensively and realistically, which is the foundation for providing reliable interpretations. To obtain the dynamic behavior trajectory, we conduct survey on popular dynamic analysis tools and summarize them in Table I. We can notice that the current available dynamic analysis tools are limited. More importantly, although these tools have achieved impressive successes in feature extraction, none of them fully meet our special requirements, such as supporting the analysis of recently released API versions and customizing the monitored APIs to automatically generate the dynamic behavior trajectories. For this reason, we formulate a new dynamic analysis tool XTracer$^+$ (available at https://github.com/kelinroland/XTracer.git) to automatically generate the dynamic behavior trajectory for each App.

By conducting extensive experiments using multiple public datasets, Fan et al. [4] found that the interpretations offered by commonly used explanation approaches are often confusing. Typically, given a known malware, the interpretations provided by different explanation approaches using same classifier are

Authorized licensed use limited to: Jain University. Downloaded on January 05,2026 at 06:29:44 UTC from IEEE Xplore. Restrictions apply.

TABLE I
EXISTING DYNAMIC ANALYSIS TOOLS

Tool	Framework	Android SDK	Batch processing	API customization	Sequential behavior	Open source
DroidBox [27]	Xposed	Android 4.1.1 (16)	○	○	◐	●
CuckooDroid [28]	Xposed	Android 4.1.2 (16)	◐	○	◐	●
APIChecker [29]	Xposed	Android 8.1 (27)	●	○	○	○
MobSF [30]	Frida	Android 9 (28)	◐	◐	●	●
XTracer+ (ours)	Frida	Android 11 (30)	●	●	●	●

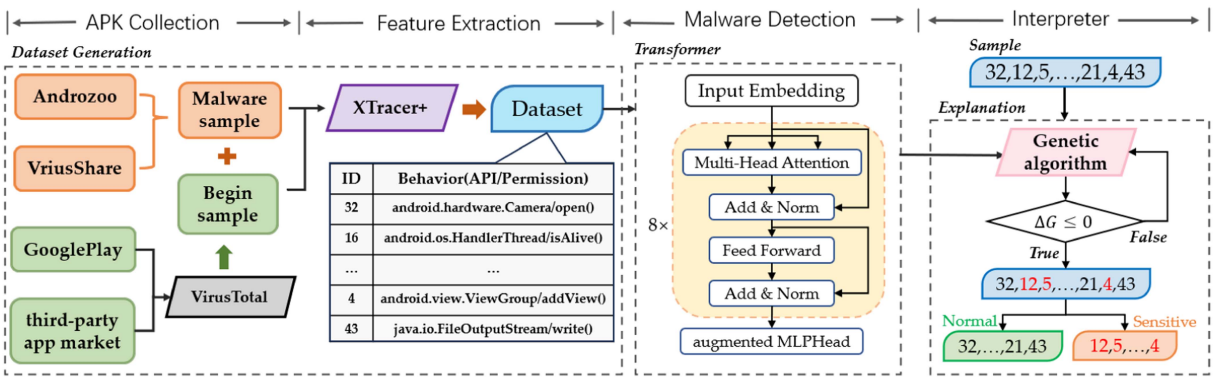

Fig. 1. The comprehensive overview of the proposed framework XDeepMal.

quite different. For instance, in [4], For LIME [6], the feature READ_SMS has played a positive role in predicting a malicious label, while for SHAP [7], it has had a negative impact. In addition to being plagued by approximate representations of static features, existing explanation approaches primarily focus on revealing how individual features impact model predictions. We argue that explanation methods that only consider the importance of individual features can easily lead to confusing interpretations, as a malicious behavior is often caused by multiple consecutive actions [4], [16].

To obtain reliable interpretations, unlike existing explanation methods that mainly focus on the contribution of individual features, we probe the suspicious fragments of dynamic behavior trajectories that have the most significant impact on predictions. It is intuitive to select the sub-sequences with the highest corresponding fitness values as the most suspicious behavior fragments. Thus, the identification of the most suspicious behavior fragments is regarded as an optimization problem. In this work, we employ a heuristic algorithm [21] to obtain an approximate global optimal solution.

III. THE PROPOSED METHOD XDEEPMAL

The main objective of this work is to propose a novel explanation framework, named as XDeepMal, for deep learning-based malware detection models. This framework is powered by dynamic analysis and aims to uncover underlying reasons why an app is predicted as malware. Fig. 1 provides an overview of the XDeepMal workflow. Specifically, it mainly contains three parts: (1) A dynamic analysis tool XTracer$^+$ is proposed to generate the dynamic behavior trajectory for each app; (2) A sequential model based on Transformer [19] is formulated for malware detection; (3) An interpreter that incorporates a heuristic algorithm [21] is raised to pinpoint the most suspicious behavioral fragments.

A. XTracer$^+$ Tool

To obtain the dynamic behavior trajectories of Android apps, we formulate a lightweight dynamic analysis tool XTracer$^+$ based on Frida [45], which injects scripts into the target app to hook functions or spy on crypto APIs. XTracer$^+$ monitors the specified API calls of each app, and records their invocation order to automatically generate the behavior trajectory. It is obviously impossible to monitor all APIs, as it will sharply increase the analysis time [29]. Therefore, referring to some time-tested tools (e.g., APIChecker [29], CuckooDorid [28] and Droidmon [46]), we have identified a total of 479 important APIs for monitoring, which are available at https://github.com/kelinroland/XTracer.git.

In recent years, reflective invocation has emerged as a popular means of accessing sensitive APIs with malicious intent. Simply tracking APIs in a straightforward manner may miss the underlying purpose concealed within malware. Therefore, it is crucial to monitor reflectively invoked APIs. Sun et al. [47] highlighted that reflective calls often follow a specific pattern and utilize reflective methods such as getName(), getMethod(), forName(), and invoke. Therefore,

Authorized licensed use limited to: Jain University. Downloaded on January 05,2026 at 06:29:44 UTC from IEEE Xplore. Restrictions apply.

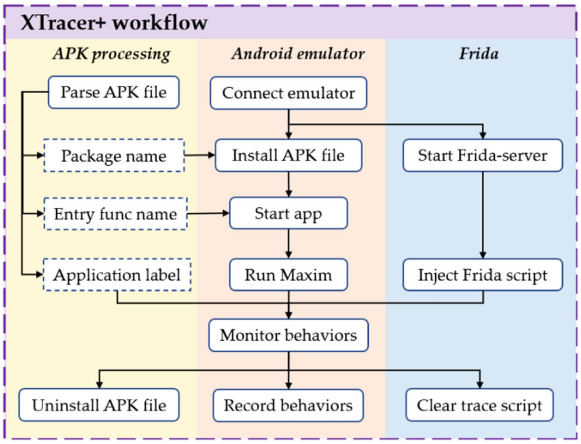

Fig. 2. The interactive workflow of the formulated XTracer$^+$.

in XTracer$^+$, we track the commonly used reflective methods and compare their input and return values to obtain the APIs invoked reflectively. For another category of advanced semantic feature permissions, normal permissions are automatically granted upon app installation [48]. On the other hand, sensitive permissions are typically requested during program execution when their corresponding sensitive APIs are invoked. For instance, certain permissions can be requested via ActivityCompat.requestPermissions() [49]. Consequently, we can trace these API requests to monitor the usage of sensitive permissions.

After incorporating the above crucial operations into the Frida script, we implement an automation script in Python to enable batch sample testing. The workflow of XTracer$^+$ is shown in Fig. 2. Given an app, its detailed execution process in XTracer$^+$ is as follows.

- Android Debug Bridge (adb) is used to connect the host and Android emulator. Specifically, the Android Asset Packaging Tool (aapt) is used to parse the APK file and obtain the certain information (e.g., package name, Entry Point Function Name).
- The APK file is installed in the emulator using the adb command and it is launched using the obtained entry point function.
- The Frida-server service is started on the emulator and the Frida script is deployed to monitor important behaviors of the target app.
- Maxim [50], a high-performance Android Monkey based on traversal rules, is executed, to automatically generate abundant triggering events to improve execution path coverage.
- Keeping synchronized with Maxim, the triggering threads and sequences of important behaviors being tracked are recorded to generate the behavior trajectory.
- The target app is uninstalled and its running traces are cleared.

B. Malware Detection Model

Transformer [19] has achieved impressive success in Natural language processing (NLP) [51]. It relies on self-attention mechanisms to capture context information from the entire sequence and allows parallel computation, which is highly effective in modeling semantic relationships [52]. Therefore, a transformer-encoder and an augmented MLPHead are integrated together to construct our malware detection model. The transformer encoder consists of 8 identical blocks, each block composed of a multi-head attention module and a fully connected feed-forward module. The augmented MLPHead is composed of an MLP with two linear layers and a residual channel. This design enhances the model capability to capture nonlinear representations while effectively preventing network degradation.

The behavior trajectory captured by XTracer$^+$ serves as the input for our Transformer based detection model. However, these behavior trajectories comprise sequential features of varying lengths. It is evident that some APIs, like android.webkit.WebView.getVisibility(), are being invoked repeatedly, which has ultimately resulted in the emergence of particularly extensive sequential features. As we know, to ensure a trade-off between performance and computational costs, transformer based models often enforce restrictions on input lengths [53]. Therefore, there is a need to standardize the length of these sequence features. It is worth noting that direct truncation or segmentation may result in the loss of crucial information. Without loss of generality, to manage features that surpass the typical capacity of 512 tokens in the Transformer model [54], we devise two strategies. The first one involves regarding continuous repeated invocations as a single one. The second one involves retaining the initial k occurrences of each invocation, starting from the midpoint of the sequence. By adjusting the value of k, we have the flexibility to regulate the sequence length to accommodate the requirements of the detection model. On the contrary, some captured sequential features are exceptionally short (i.e., less than 14), making it difficult to capture discriminative behaviors. Therefore, XDeepMal designates apps with these shorter sequential features as failure analysis samples, which is a method commonly used in this field [55]. The manual investigations conducted by Cui et al. [55] have also highlighted that repeated benign paths or actions, and short sequence features are important contributing factors to the problem of false negatives.

C. Interpreter

To establish user (e.g., security analyst) trusts in prediction decisions yielded by our Transformer-based malware detection model, we propose an explanation approach to point out why an app is predicted as malware. Our interpreter incorporates insights from recent studies on ablation analysis, such as Ablation-CAM [56]. The foundation of this work lies in the observation that removing certain feature units has a significant impact on the confidence score of particular classes. The extent of the decrease can serve as an indicator of the importance level of those feature units for the respective classes [56], [57]. The proposed interpreter is capable of handling input sequences with varying lengths. As an example, the preprocessed behavior sequence A of an app is fed into the trained Transformer based detection model for malware detection, which can be illustrated as follows:

$$y = f(A) \tag{1}$$

Authorized licensed use limited to: Jain University. Downloaded on January 05,2026 at 06:29:44 UTC from IEEE Xplore. Restrictions apply.

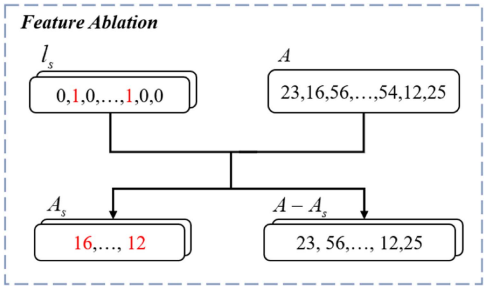

Fig. 3. The process of the feature ablation.

where f represents the detection model that takes a sequence as input and produces a prediction value for a given app. Let's assume that A_s is a random sub-sequence of input sequence $A$and y is the prediction score when A_s is present. The elements of A_s are ablated from $A$and the sequence after ablation is passed the model to get a (possibly) reduced confidence score y_s as follows:

$$y_s = f\left(A - A_s\right) \tag{2}$$

The evaluation function illustrating the effect of ablation of A_s is given by following:

$$g\left(A_s\right) = y - y_s \tag{3}$$

However, as the length of A_s increases, $g(A_s)$ usually rises as well. Consequently, the maximum of g often corresponds to a long sequence A_s, which can lead to excessively long interpretation results. However, such outcomes cannot be considered reliable or conclusive interpretations. Hence, it is necessary to impose constraints on the length of A_s, which results in the formulation of the following objective function.

$$\begin{aligned} &\max_{A_s} \ \ln\ g\left(A_s\right) \\ &s.t. \frac{|A_s|}{|A|} < C \end{aligned} \tag{4}$$

where C represents the requirement that the length ratio of A_s to A should be lower than a specific threshold. To reduce the complexity of our algorithm, we convert the constraint problem to unconstraint by maximizing the Lagrange function with fixed Lagrange multiplier λ:

$$\max_{A_s} \ \ln\ g\left(A_s\right) - \lambda \frac{|A_s|}{|A|}, (\lambda > 0) \tag{5}$$

The obtained value can be simply interpreted as the maximum decrease in the confidence score of class c (malicious or benign) when $A_s$is removed, while the number of elements in A_s is relatively small. That is, we can identify the sub-sequences of the behavior trajectories that exert the most significant impact on the model predictions. To facilitate intuitive understanding, we present a simple example to obtain A_s in Fig. 3. In Fig. 3, l_s is introduced as a masking layer applied to A, where 0 means feature preservation and 1 means feature removal at the corresponding position of the sequence$A$. Based on this, the optimization problem for solving A_s as described in (5) is transformed into a problem for solving l_s.

The genetic algorithm [21] is a well-known heuristic optimization algorithm that draws inspiration from principles found in natural selection, genetics and evolution. It is a powerful and versatile search approach that efficiently converges to identify the optimal solution within a large solution domain exploiting genetic manipulations [58]. Therefore, we exploit the genetic algorithm to discover our optimal solution. The parameters utilized in our genetic algorithm are detailed in Section IV.

It is worth mentioning that the objective function in (5) also serves a fitness function. The objective is to strike a balance between the effectiveness of explanations and the length of the interpretations. As a result, individuals with higher fitness values represent more reliable explanation outcomes. In each generation, we begin with the selection operation, followed by crossover and mutation, to generate new individuals. These new individuals are then evaluated based on the fitness function, and the optimal individuals are selected to form the population for the next generation.

IV. Experiment

A. Dataset

Building a representative malware dataset is a challenging task that can be executed in various ways, depending on the data requirements and the analysis objectives [18]. Our goal is to discover why an app is predicted as malware in a deep learning-based malware analysis pipeline, particularly in real-world scenarios where numerous samples from various sources are analyzed regularly. Thus, we downloaded samples from various sources, encompassing the Google Play Store [23], Androzoo [25], [26], VirusShare [24], and third-party app markets, to minimize the risk of data distortion from relying on a single source. To mirror real-life scenarios and enhance the reliability of the dataset, the process of downloading malware involves randomly selecting fresh samples from multiple sources to gather as many different types of malware as possible. It is important to note that as new types or variants of malware continually emerge, it is difficult for the dataset to cover all possible types. Regarding benign apps, as depicted on the left side of Fig. 1, we keep those identified as benign by all virus scanners in VirusTotal [50]. After the de-duplication process of apps, we obtain a dataset of 6692 benign apps and 7286 malicious apps. Following a rigorous dynamic analysis process, we have successfully scrutinized 2842 benign apps and 2946 malicious apps, respectively. Employing a random partition, 80% of these instances are employed for training, while the remaining 20% constitute the test set. It is noteworthy that all experimental findings in this study are derived from the test set.

B. Parameter Settings

Similar to the standard transformer structure, the encoder in our transformer based detection model consists of a stack of N = 8 identical layers. Each layer comprises two sub-layers: a multi-head self-attention mechanism and a position-wise fully connected feed-forward network. The number of heads in the self-attention mechanism is set to 8. The length of the input

Authorized licensed use limited to: Jain University. Downloaded on January 05,2026 at 06:29:44 UTC from IEEE Xplore. Restrictions apply.

sequence is 512, and the value of k is set to 5 in the sequence preprocessing stage. It is worth noting that the value of k can be dynamically adjusted according to the length of the processed sequence. We employ an embedding layer to convert the input sequences into vectors with the dimension d_m, which is set to 768. To facilitate the utilization of residual connections in the transformer, all sub-layers in the encoder also generate outputs with the dimension 768. The Adam optimizer is used in this work, and the batch size is set to 64. The value of learning rate and epoch is set to 0.0001 and 100, respectively. As a genetic algorithm has been utilized in our interpreter, its parameter settings are as follows. The population size is set to 40, the crossover probability is set to 0.5 and the mutation probability is set to 0.1. For the termination criteria, we set the maximum generations to 200. Specifically, the population is initialized by randomly generating 40 individuals. For the selection process, we utilize a roulette wheel selection strategy, where the selection probability of each individual is directly proportional to its fitness. As for the crossover operation, we employ a double-point crossover method. Two individuals are randomly selected, and the genes at two random points between these individuals are exchanged.

C. Evaluation Metrics

It is reasonable to assume that the effectiveness of explanation results is directly proportional to their discernible impact on the classification results of a given app. Hence, we introduce a new measure to assess the effectiveness of our explanation method. Let us consider an app with sequential features represented as x_i(e.g., $x_i = \{0, 481, 479, 434, 436, \ldots, 430\}$) belonging to class c(benign or malicious). The most suspicious behavioral fragment, denoted as I_i(e.g., $I_i = \{0, 481, 479\}$), represents the corresponding interpretations of our interpreter. When the interpretation I_i is removed from x_i, a modified sequential feature denoted as $\overline{x_i}$ (e.g., $\overline{x_i} = \{434, 436, \ldots, 430\}$) can be obtained. The modified one can then be fed into the trained detection model, resulting in a new prediction value denoted as $\overline{y_c}$. This value $\overline{y_c}$ is then juxtaposed with the prediction value y_c of the original sequential feature x_i. The degree of change in this prediction value serves as a metric to assess the effectiveness of the interpretation I_i. Therefore, the formula for calculating effectiveness is as follows:

$$E_i = y_c - \overline{y_c} \tag{6}$$

where y_c and $\overline{y_c}$ correspond to the model predictions for class c. A larger magnitude of variation of E_i indicates a greater impact of the interpretation result I_i on the classification of the app.

D. Comparison With Baseline Methods

To comprehensively evaluate the effectiveness of XDeepMal, it is essential to establish a widely recognized baseline. Notably, the detection model in XDeepMal is Transformer-based, but the explanation method is a local and model-agnostic approach. Fan et al. [4] collected five widely-used malware datasets and conducted extensive experiments to assess the performance of five prominent local and model-agnostic explanation approaches (i.e., LIME [6], Anchor [59], LORE [60], SHAP [7]

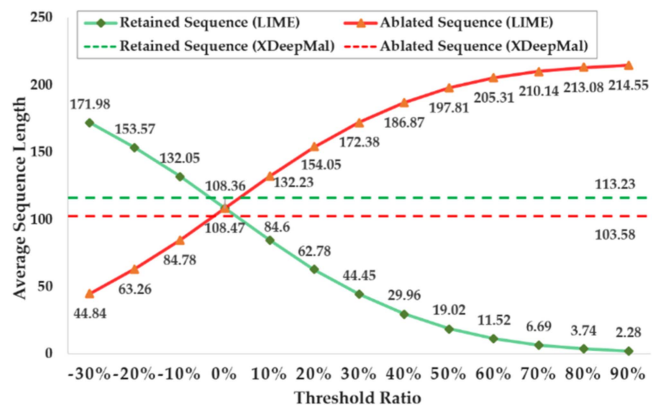

Fig. 4. Average Retained Sequence Length and Average Ablated Sequence Length for Different Threshold Ratios for LIME, Compared to XDeepMal.

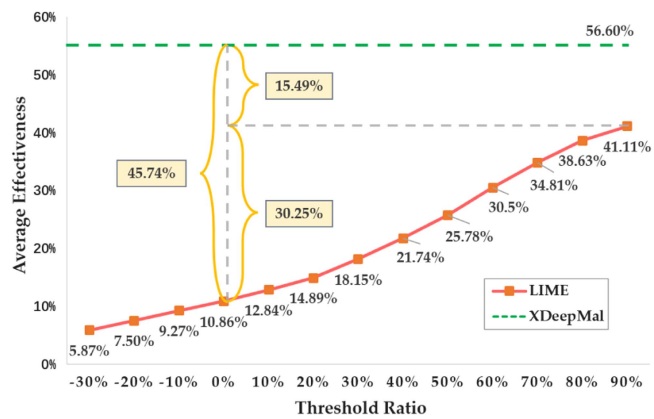

Fig. 5. Average effectiveness variation of LIME at different threshold ratios, compared to XDeepMal.

and LEMNA [61]) in the context of Android malware analysis, using a variety of machine learning (including deep learning) based classifiers. Aligning with the findings presented in [4] and [40], we select LIME [6] as our baseline method and don't consider the other model-agnostic methods explored in Fan et al. [4], as the experimental results indicate that LIME provides superior interpretations for learning based malware detection. Although the output weight values in LIME are partitioned based on a threshold of 0 to distinguish their magnitudes, larger weights typically indicating benign behaviors and smaller weights signifying malicious behaviors, this approach has been validated in various domains.

However, it can be challenging to fairly assess the performance of LIME using a single arbitrary threshold. Therefore, we adjust the threshold in our comparative experiment to thoroughly evaluate the effectiveness of LIME. Due to the varying magnitudes of weight values for each app, it is not feasible to use a fixed value to uniformly partition all apps. Therefore, we employ a method based on the maximum weight ratio for partitioning. Specifically, we select the largest absolute weight value for each app in our test set, denoted as w, and then adjust the threshold ratio $Tr \in (-100\%, 100\%)$ to generate the thresholds with $T = Tr \times w$. As depicted in Figs. 4 and 5, the threshold ratio adjustment interval is set at 10%. However, to better represent the

Authorized licensed use limited to: Jain University. Downloaded on January 05,2026 at 06:29:44 UTC from IEEE Xplore. Restrictions apply.

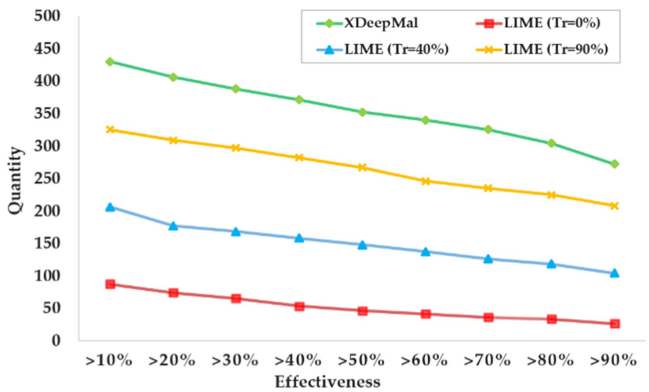

Fig. 6. The quantity of apps within various effectiveness categories in our test set.

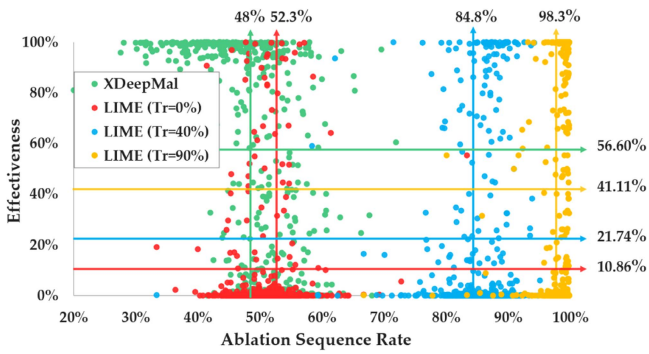

Fig. 7. The relationship between the effectiveness of the interpreter outputs and the ablation sequence rate on the test set.

actual context corresponding to the interpretations, we display a range of -30% to 90%.

In this subsection, the interpretations achieved from LIME indicate that features with weights below the threshold T contribute to predictions of the malicious class, while features with weights above the threshold T contribute to predictions of the benign class. Fig. 4 illustrates the average lengths of interpretation results obtained at different Tr, while Fig. 5 presents the average effectiveness across various Tr. Fig. 6 depicts the quantity of apps within various effectiveness categories in our test set. Meanwhile, Fig. 7 showcases the relationship between the effectiveness of the interpreter outputs and the ablation sequence rate (i.e., the ratio of the length of the interpretation to the total length of the sequence) on the test set.

E. Quantitative Analysis

In Section IV.D, we conduct experiments to quantitatively compare and analyze the interpretations of XDeepMal with those of LIME. Specifically, we focus on two aspects: the effectiveness and the length of interpretations. We present the experimental results from both a holistic app perspective and an individual app perspective. The results from this comparative study emphasize that our XDeepMal strikes a balance between maintaining satisfactory effectiveness measures and rationalizing the length of interpretations. This balance leads to more concise and reliable interpretation results, thus enhancing the overall interpretability of the model.

When evaluating the length of interpretations, it can be observed from Fig. 4 that the average length of interpretations generated by XDeepMal closely resembles that of LIME when a 0% threshold ratio is used. In comparison to the length of ablated sequences (i.e., the interpretations) produced by LIME at a 0% threshold ratio, XDeepMal reduces this average length by 4.89. We can also notice that as the threshold ratio increases, the length of interpretations obtained by LIME also tends to increase as well.

In terms of the effectiveness of interpretations, as illustrated in Fig. 5, XDeepMal demonstrates an average effectiveness rate of 56.60%, while LIME achieves only 10.86% at the 0% threshold ratio. For LIME, as the threshold ratio increases, the effectiveness of the interpretations improves. The best performance is observed at a 90% threshold ratio, where the average effectiveness reaches 41.11%. This represents a substantial improvement of 30.25% compared to the default 0% threshold ratio. However, XDeepMal still outperforms LIME by 15.49%. It is worth noting that the observed improvement in effectiveness, which amounts to 30.25% when transitioning from a 0% to a 90% threshold ratio, is accompanied by a notable increase in the length of interpretations for LIME.

Figs. 6 and 7 provide a comparative analysis of XDeepMal and LIME across different threshold ratios (i.e., 0%, 40%, and 90% ratios). These figures offer insights from two distinct perspectives: the overall distribution of apps and the individual distribution of apps. Fig. 6 shows that XDeepMal maintains a high level of effectiveness across most apps, even when compared to LIME using a threshold ratio of 90%. From Fig. 7, we can observe that when LIME is used with the default threshold ratio of 0%, XDeepMal and LIME demonstrate similar average lengths of interpretation results, accounting for 48% and 52.3%, respectively. However, a notable discrepancy arises in their average effectiveness metrics, with XDeepMal achieving 56.60% effectiveness while LIME achieves only 10.86%.

The aforementioned quantitative comparison and analysis confirm that our proposed explanation framework, XDeepMal, holds several advantages over the classic universal method LIME in terms of the effectiveness and length of interpretations. Generally, XDeepMal exhibited the ability to generate concise and accurate interpretation results.

F. Real-world Case Study

In this section, we conduct real-world case studies, including two specific malware families (SMSreg and SMSpay) and individual apps, to evaluate the explanation performance of XDeepMal. To implement case studies on malware families, we first identify family labels for the malicious apps in our test set utilizing [62]. However, the malicious behaviors exhibited by certain malware can be complex and diverse, causing them to belong to multiple families simultaneously. This poses a challenge in using such a mixed behavior as a fair benchmark for evaluation purposes. Therefore, we selected the SMSreg family, known for its clear behaviors, as benchmark to analyze

Authorized licensed use limited to: Jain University. Downloaded on January 05,2026 at 06:29:44 UTC from IEEE Xplore. Restrictions apply.

TABLE II
THE NUMBERS CORRESPOND TO SPECIFIC BEHAVIORS (CORRESPONDS TO SMSREG)

Behavior category	Index	Fragments	Behavior
Encryption	470	470, 471	javax.crypto.Cipher/getInstance
decryption	471	470, 471	javax.crypto.Cipher/init
Operate file	43		java.io.FileOutputStream/write
	298		java.io.FileNotFoundException/toString
	346	434, 346	java.nio.charset.CoderResult/isError
	430		java.io.File/delete
	434	434, 346	java.io.File/renameTo
	479		WRITE_EXTERNAL_STORAGE
Device info	0		android.telephony.TelephonyManager/getDeviceId
	6	6, 492	android.net.wifi.WifiInfo/getMacAddress
	481		ACCESS_FINE_LOCATION
	492	6, 492	RECEIVE_SMS
Reflection call	28		java.lang.reflect.Method/invoke
	411	411, 413	android.app.Instrumentation/callActivityOnPostCreate
	413	411, 413	android.app.Instrumentation/callActivityOnStart
	460		java.lang.Class/forName

the reliability of the interpretations provided by XDeepMal. For instance, a common malicious behavior exhibited by SMSreg is the illegal acquisition of SMS verification codes from infected devices, which may be sent to attackers, allowing them to bypass authentication measures and gain unauthorized access. To evaluate the explanation generality of XDeepMal, we also conduct the evaluation of XDeepMal on the SMSpay family. Notably, the SMSpay family samples discussed here are labeled with both SMSreg and SMSpay. The SMSpay family is recognized for its malicious behaviors, including sending premium-rate or spam SMS messages without users' consent, resulting in unexpectedly high phone charges. Furthermore, it can serve as a medium for phishing attacks and other fraudulent activities, leading to serious financial losses [63], [64]. It is worth noting that in this work, the class label 0 corresponds to malicious apps, while the class label 1 corresponds to benign apps. The model detection results mentioned in the subsequent sections specifically refer to predictions made for class 0, which represents the malicious app class.

Given an app (with file code 238f70c0061a358db21d5 09c2744b3b18853428d9d0c1f9b17c767d1d4bd186e) from the SMSreg family, the interpretation obtained by XDeepMal is presented in Fig. 8. The length of the feature sequence is 110, and the specific behaviors represented by the sequence numbers can be obtained from Table II. Since the obtained interpretation only covers a specific segment of the feature sequence, namely $\{0, 481, 479\}$, many other parts of the sequence are not shown in Fig. 8. The whole sequence features and the sequence positions of the interpretation can be referenced in Appendix A.1, available online and Appendix A.2, available online, for a comprehensive understanding. From Fig. 8, we can notice that the model predicts the app as malicious with a value of $y_0 = 0.9954$, but after eliminating the sequence features $\{0, 481, 479\}$, the probability of predicting it as malicious only decreased to $\overline{y_0} = 0.0152$. However, we can also observe that if single ablation for the

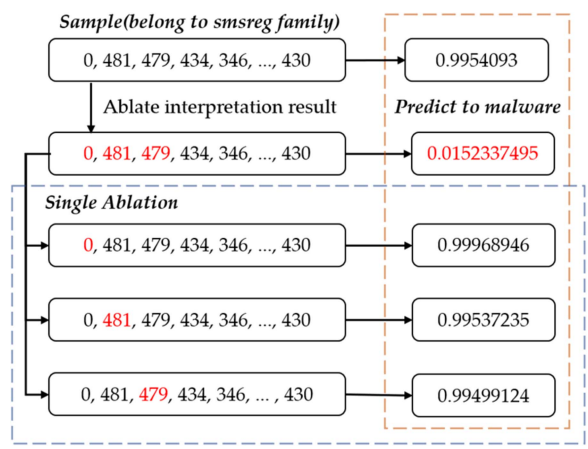

Fig. 8. The evaluation of XDeepMal interpretations for a given app.

individual feature is performed, the predictions yielded by the Transformer base detection model still tend to be malicious. The probability of predicting it as malicious even increased when the 0 feature (android.telephony.TelephonyManager/getDeviceId) is removed individually.

When the explanation framework LIME [6] is used to calculate the weight (the whole weight calculation result can be found in the Appendix A.3, available online), the weights of 0, 481 and 479 are 0.059423, 0.002328 and -0.013037, respectively. In the entire interpretation result achieved by LIME [6], feature 0 has the highest weight of 0.059423. By setting the threshold to 0, it becomes evident that feature 0 is the most significant benign feature. This observation aligns with the experimental results obtained from our individual ablation analysis for feature 0 in Fig. 8. However, upon analyzing the interpretation provided by XDeepMal, it can be found that even benign features, when combined with other features, may have a significant impact on the model decision-making process. This is mainly due to the

Authorized licensed use limited to: Jain University. Downloaded on January 05,2026 at 06:29:44 UTC from IEEE Xplore. Restrictions apply.

Fig. 9. Behavior pattern analysis of the SMSreg malware family.

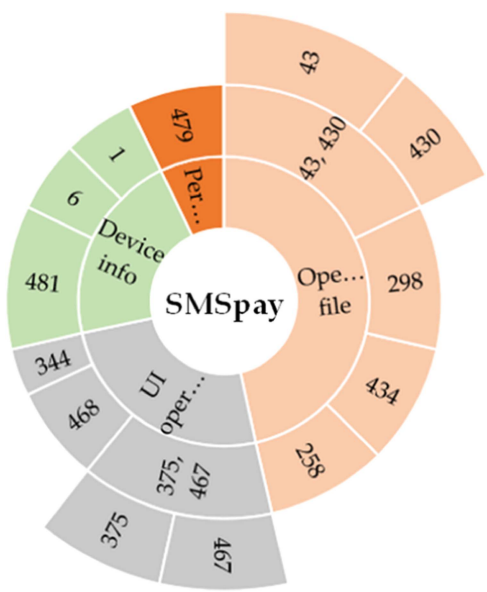

Fig. 10. Behavior pattern analysis of the SMSpay malware family.

fact that the implementation of a malicious behavior typically requires a series of consecutive actions. This is not an isolated phenomenon, as the continuous sequences of numerous other apps also play a vital role in the prediction process of the model. The analysis of the behavior patterns exhibited by the malware family in the subsequent text further reinforces this opinion.

In addition to evaluating individual apps, the behavior patterns of two specific malware families are calculated by analyzing the interpretation results of multiple apps within the families. The occurrence frequencies of each behavior fragment are counted, and the single or multiple fragments with a higher frequency are used to form a representative behavior pattern for the two malware families. Figs. 9 and 10 illustrate the behavior patterns of the SMSreg and SMSpay malware families, respectively. As shown in Fig. 9, the behavior pattern of the SMSreg family includes receiving and sending SMS by obtaining RECEIVE_SMS permission, reading and modifying files in storage devices using java.io.FileOutputStream/write(), and collecting device information like IMEI using android.telephony.TelephonyManager/getDeviceId. These functionalities exhibit significant similarities to those observed in the SMSreg family within our practical applications, as they all involve these core behaviors. Additionally, the SMSreg family shares similarities regarding certain behaviors, including the use of encryption and reflection techniques. These behaviors enable the family to conceal its malicious nature and bypass security checks effectively. For the SMSpay family, in addition to file operations and obtaining device information, we also notice its suspicious behavior, including UI operations. In particular, these UI operations, such as android.widget.TextView/setText and android.view.ViewGroup/addView, may provide opportunities for implementing phishing attacks. For instance, attackers might manipulate the setText method to trick users entering sensitive information, and they can also use addView to overlay a fake form on top of a legitimate app screen.

In summary, from Tables II and III, the functionality of the malware family obtained through XDeepMal closely resembles the inherent features of SMSreg and SMSpay, particularly in terms of its core behavior. Therefore, it can be inferred that the interpretations provided by the proposed method, XDeepMal, distinctly exhibit the characteristics of SMSreg and SMSpay. Through a detailed analysis of the interpretations of the SMSreg and SMSpay malware families, although it may not cover all malicious behaviors, it provides valuable insights into the pattern analysis of the overall family classification and the behavior trajectory analysis of individual apps that adhere to real-world behavioral logic. Thus, we believe that the proposed explanation framework XDeepMal can provide reliable interpretations for the decisions made by the deep learning-based malware detection model. This is achieved by dynamically modeling the continuous behavior trajectories of apps and combining heuristic algorithms together to obtain the crucial behavior fragments.

G. Coverage and Efficiency

As mentioned above, in the XDeepMal framework, a dynamic analysis tool XTracer$^+$ is proposed to manufacture the dynamic behavior trajectory for each app. *Is XTracer$^+$ able to achieve higher activity coverage to capture crucial behaviors? Does XDeepMal show feasible inference time?*

In XTracer$^+$, an Monkey-based tester Maxim [50] is used to generate UI action events to simulate interactive behavior, covering a wider range of app activities (aligned with the app's key functionalities) [65]. The implementation of these core functionalities involves a series of API calls, forming the raw resource of our dynamic behavior trajectory. Thus, we use a popular UI testing coverage metric presented in [65], [66] to measure the activity coverage. This coverage metric is calculated by $exploredActivity/totalActivity \times 100\%$, where exploredActivity represents the activities that have been covered

Authorized licensed use limited to: Jain University. Downloaded on January 05,2026 at 06:29:44 UTC from IEEE Xplore. Restrictions apply.

TABLE III
THE NUMBERS CORRESPOND TO SPECIFIC BEHAVIORS (CORRESPONDS TO SMSPAY)

Behavior category	Index	Fragments	Behavior
Permission request	479		WRITE_EXTERNAL_STORAGE
Operate file	43	43,430	java.io.FileOutputStream/write
	258		java.nio.charset.CoderResult/isOverflow
	298		java.io.FileNotFoundException/toString
	430	43,430	java.io.File/delete
	434		java.io.File/renameTo
Device info	1		android.telephony.TelephonyManager/getSubscriberId
	6		android.net.wifi.WifiInfo/getMacAddress
	481		ACCESS_FINE_LOCATION
UI operations	344		android.widget.EditText/setFocusable
	375	375,467	android.widget.LinearLayout/setBackgroundDrawable
	467	375,467	android.view.ViewGroup/addView
	468		android.widget.TextView/setText

and totalActivity is retrieved via the framework's PackageManager.getPackageInfo interface, which implies it may encompass activities that have been abandoned, made invisible, or are not-reachable [66]. In order to assess the test effectiveness of XTracer$^+$, we select the apps from the SMSreg malware family (i.e., Group 1) and SMSpay (i.e., Group 2) as examples, calculating their average activity coverage, respectively. The average activity coverage achieved by Group 1 and Group 2 are 18.02% and 12.93%, respectively.

It is apparent that there is scope for enhancement in this activity coverage rate. Following a detailed analysis, we argue that the calculation method for the totalActivity can contribute to lower coverage, and that factors such as app execution time are also correlated. Although long-time execution time can help to cover a wider scope of activities, it unquestionably entails substantial resource consumption. To cater to the demands of large-scale sample detection in real-world scenarios, the runtime utilized in this work is 2 minutes, which aligns with the rigorous experimental verification performed by Kuchler et al. [18]. Significantly, most of the behaviors were observed during the initial 2-minute period [18]. Our experiments in Section IV.F can also provide evidence that, even under the given coverage restrictions, it is attainable to realize interpretable performance that is more consistent with the real-world scenario.

During the practical deployment of XDeepMal, the analysis experts take the dynamic behavior trajectories of the apps captured by our XTracer$^+$ as inputs to a Transformer-based detection model that has been trained beforehand, enabling both malware detection and interpretation of Decision Basis. In [18], "*an analyst needs to find the 'sweet spot' that allows to collect only the minimum amount of information required to properly classify each sample*.". As mentioned above, the 'sweet spot' in this work is set to 2 minutes. Hence, the key focus here is on evaluating the inference time of the explanation scheme. In this part, we further take the samples of two groups (i.e., the SMSreg and the SMSpay malware family) as illustrative examples to compute their average inference time. To match the daily work conditions of analysts, the hardware environment we have deployed is "CPU: Intel(R) Core(TM) i5-8300H CPU @ 2.30GHz and RAM: 16GB". The average inference time is 156.79 seconds for Group 1 and 232.25 seconds for Group 2. The observed inference time is considered reasonable in comparison to manual analysis. Additionally, continued improvements to the hardware environment can enable a persistent reduction in this inference time.

H. Limitation and Discussion

Despite the promising performance of XDeepMal, it still suffers from some potential limitations. First, while dynamic analysis proves more effective than static analysis in handling code obfuscation, dynamic code loading, and reflection, evasion techniques persistently evolve and shift. Just as other dynamic analysis-based methods, XDeepMal may also be targeted by malware developers attempting to evade analysis by using more sophisticated evasion techniques. Although this phenomenon is common, we make efforts to conservatively filter out samples that did not run correctly in our XTracer$^+$ environment, particularly those with shorter sequential features. Second, there are feature extraction failures in certain apps. For instance, as the number of hooked APIs increases, the success rate of app dynamic analysis tends to decrease accordingly. However, when reducing the number of hooked APIs, there is a possibility of omitting significant behaviors, which can impose limitations on the effectiveness of malware detection. To tackle this challenge, we thoroughly examined various time-testing methods and identified 479 sensitive API calls. Our experimental results, supported by efficient analysis and real-world case studies, confirmed the effectiveness of these APIs. Besides, Our XTracer$^+$ tool supports users to customize the monitored APIs, automatically generating dynamic behavior trajectories. Third, the quality of interpretations will be limited by the accuracy of detection models based on deep learning. Like other deep learning-based methods, the performance of our detection model largely depends on the quality of the training set. As we know, it is difficult to enumerate all possible types and variants of malware

Authorized licensed use limited to: Jain University. Downloaded on January 05,2026 at 06:29:44 UTC from IEEE Xplore. Restrictions apply.

in the training set, which poses a common challenge for such learning-based detection models. Despite our efforts to align the constructed dataset with real-world application scenarios and diversify malware types, it remains insufficient for addressing constantly emerging new malware. Therefore, ongoing efforts are essential in the field of malware detection. Besides, inspired by the remarkable advantages of Transformer in effectively modeling long-term contextual correlations, XDeepMal adopted the Transformer-based detection model. The primary objective of this study is to provide reliable interpretation, which is a model-agnostic method. This means the Transformer model can be replaced, allowing for the introduction of new Transformer variants. In the future, we will prioritize improving the processing of sequential features and optimizing our detection model.

V. Conclusion

In this work, we proposed a novel framework XDeepMal to interpret deep learning-based malware detection models to understand why an app is classified as malware. Specifically, to enhance the reliability of interpretation results, we formulated a lightweight dynamic analysis tool called XTracer$^+$. This tool, available at https://github.com/kelin-roland/XTracer, has the capability to automatically generate behavior trajectories, allowing for precise modeling of the continuous runtime behaviors exhibited by apps. Extracted dynamic behavior trajectories are fed into our Transformer-based detection model for malware detection. To establish user trusts in the predictions, we proposed a new interpreter to pinpoint the malicious behavior fragments that plays the most critical role in model decision-making. Finally, we performed a quantitative comparison and analysis, evaluating the effectiveness of the proposed framework XDeepMal against the classic LIME scheme under diverse conditions. Our experimental results clearly demonstrated that XDeepMal consistently exhibits superior average effectiveness compared to LIME. Besides, to assess the reliability of the interpretations obtained by XDeepMal, we conducted in-depth analysis on real case studies, encompassing individual apps and malicious families. Our experimental results reveal a significant level of consistency between the identified suspicious dynamic behavior fragments and the actual observed behaviors. Finally, we assessed its coverage and inference time to determine its feasibility for real-world deployment. These findings provide strong evidence that XDeepMal, which extracts dynamic continuous behavior trajectories and employs heuristic algorithms, is an effective approach for obtaining reliable interpretations.

Acknowledgment

The authors would like to thank the anonymous reviewers for their valuable comments and feedback to improve our paper.

References

[1] Y. Liu, C. Tantithamthavorn, L. Li, and Y. Liu, "Deep learning for android malware defenses: A systematic literature review," *ACM Comput. Surv.*, vol. 55, no. 8, pp. 1–36, 2023.

[2] S. Chen, Y. Zhang, L. Fan, J. Li, and Y. Liu, "AUSERA: Automated security vulnerability detection for android apps," in *Proc. 37th IEEE/ACM Int. Conf. Automated Softw. Eng.*, 2022, pp. 1–5.

[3] J. Qiu, J. Zhang, W. Luo, L. Pan, S. Nepal, and Y. Xiang, "A survey of android malware detection with deep neural models," *ACM Comput. Surv.*, vol. 53, no. 6, pp. 1–36, 2020.

[4] M. Fan, W. Wei, X. Xie, Y. Liu, X. Guan, and T. Liu, "Can we trust your explanations? Sanity checks for interpreters in android malware analysis," *IEEE Trans. Inf. Forensics Secur.*, vol. 16, pp. 838–853, 2021.

[5] T. Speith, "A review of taxonomies of explainable artificial intelligence (XAI) methods," in *Proc. ACM Conf. Fairness Accountability Transparency*, 2022, pp. 2239–2250.

[6] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why should I trust you?: Explaining the predictions of any classifier," in *Proc. 22nd ACM SIGKDD Int. Conf.*, 2016, pp. 1135–1144.

[7] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 4768–4777.

[8] B. Wu et al., "Why an android app is classified as malware: Toward malware classification interpretation," *ACM Trans. Softw. Eng. Methodol.*, vol. 30, no. 2, pp. 1–29, 2021.

[9] Y. Hiromu, S. Shinnosuke, N. Reon, O. Yoshihiro, and S. Jun, "Neural malware analysis with attention mechanism," *Comput. Secur.*, vol. 87, 2019, Art. no. 101592.

[10] M. Someya, Y. Otsubo, and A. Otsuka, "FCGAT: Interpretable malware classification method using function call graph and attention mechanism," in *Proc. Netw. Distrib. System Secur. Symp.*, 2023, pp. 1–10.

[11] S. Arzt et al., "FlowDroid: Precise context, flow, field, object-sensitive and lifecycle-aware taint analysis for Android apps," in *ACM SIGPLAN Conf. Program. Lang. Des. Implementation*, 2014, pp. 259–269.

[12] E. Mariconti, L. Onwuzurike, P. Andriotis, E. D. Cristofaro, G. Ross, and G. Stringhini, "MaMaDroid: Detecting android malware by building Markov chains of behavioral models," in *Proc. 24th Netw. Distrib. System Secur. Symp.*, 2017, pp. 1–15.

[13] Y. He, Y. Liu, L. Wu, Z. Yang, K. Ren, and Z. Qin, "MsDroid: Identifying malicious snippets for android malware detection," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 3, pp. 2025–2039, May/Jun. 2023.

[14] S. Zhang, H. Lei, Y. Wang, D. Li, Y. Guo, and X. Chen, "APIMind: API-driven assessment of runtime description-to-permission fidelity in android apps," in *Proc. IEEE 34th Int. Symp. Softw. Rel. Eng.*, 2023, pp. 427–438.

[15] X. Chen et al., "CruParamer: Learning on parameter-augmented API sequences for malware detection," *IEEE Trans. Inf. Forensics Secur.*, vol. 17, pp. 788–803, 2022.

[16] H. Cai, N. Meng, B. Ryder, and D. Yao, "DroidCat: Effective android malware detection and categorization via app-level profiling," *IEEE Trans. Inf. Forensics Secur.*, vol. 14, no. 6, pp. 1455–1470, Jun. 2019.

[17] Y. Liu, C. Tantithamthavorn, L. Li, and Y. Liu, "Deep learning for android malware defenses: A systematic literature review," *ACM Comput. Surv.*, vol. 55, no. 8, pp. 1–36, 2023.

[18] A. Kuchler, A. Mantovani, Y. Han, L. Bilge, and D. Balzarotti, "Does every second count? Time-based evolution of malware behavior in sandboxes," in *Proc. Int. Conf. Netw. Distrib. Syst. Secur.*, 2021, pp. 1–18.

[19] A. Vaswani et al., "Attention is all you need," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 6000–6010.

[20] Y. Fan et al., "Heterogeneous temporal graph transformer: An intelligent system for evolving android malware detection," in *Proc. 27th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2021, pp. 2831–2839.

[21] S. Mirjalili, *Evolutionary Algorithms Neural Networks: Theory Applications*. Berlin, Germany: Springer, 2019.

[22] L. Wang, H. Wang, X. Luo, and Y. Sui, "MalWhiteout: Reducing label errors in android malware detection," in *Proc. 37th IEEE/ACM Int. Conf. Automated Softw. Eng.*, 2022, pp. 1–13.

[23] G. P. Store, 2023. [Online]. Available: https://play.google.com/store

[24] VirusShare.com, 2022. [Online]. Available: https://virusshare.com/

[25] K. Allix, "AndroZoo," 2023. [Online]. Available: https://androzoo.uni.lu/

[26] K. Allix, T. F. Bissyandé, J. Klein, and Y. L. Traon, "AndroZoo: Collecting millions of android apps for the research community," in *Proc. 13th Int. Conf. Mining Softw. Repositories*, 2016, pp. 468–471.

[27] "Droidbox," 2019. [Online]. Available: https://github.com/pjlantz/droidbox

[28] "CuckooDroid," [Online]. Available: https://github.com/idanr1986/cuckoo-droid

[29] L. Gong et al., "Experiences of landing machine learning onto market-scale mobile malware detection," in *Proc. 15th Eur. Conf. Comput. Syst.*, 2020, pp. 1–14.

[30] "MobSF," [Online]. Available: https://mobsf.github.io/Mobile-Security-Framework-MobSF/

Authorized licensed use limited to: Jain University. Downloaded on January 05,2026 at 06:29:44 UTC from IEEE Xplore. Restrictions apply.

[31] Y. Chai, L. Du, J. Qiu, L. Yin, and Z. Tian, "Dynamic prototype network based on sample adaptation for few-shot malware detection," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 5, pp. 4754–4765, May 2023.
[32] J. Fairbanks, A. Orbe, C. Patterson, E. Serra, and M. Scheepers, "Identifying ATT&CK tactics in android malware control flow graph through graph representation learning and interpretability," in *Proc. 36th AAAI Conf. Artif. Intell.*, 2022, pp. 12941–12942.
[33] G. Sun and Q. Qian, "Deep learning and visualization for identifying malware families," *IEEE Trans. Dependable Secure Comput.*, vol. 18, no. 1, pp. 283–295, Jan./Feb. 2021.
[34] B. Wu et al., "Why an android app is classified as malware," *ACM Trans. Softw. Eng. Methodol.*, vol. 30, no. 2, pp. 1–29, 2021.
[35] H. Zhu, L. Wang, S. Zhong, Y. Li, and V. S. Sheng, "A hybrid deep network framework for android malware detection," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 12, pp. 5558–5570, Dec. 2022.
[36] Z. Zhang, P. Qi, and W. Wang, "Dynamic malware analysis with feature engineering and feature learning," in *Proc. 34th AAAI Conf. Artif. Intell.*, 2020, pp. 1210–1217.
[37] C. Bai, Q. Han, G. Mezzour, F. Pierazzi, and V. S. Subrahmanian, "DBank: Predictive behavioral analysis of recent android banking trojans," *IEEE Trans. Dependable Secure Comput.*, vol. 18, no. 3, pp. 1378–1393, May/Jun. 2021.
[38] A. Desnos, "Androguard-reverse engineering, malware and goodware analysis of Android applications," 2023. [Online]. Available: http://code.google.com/p/androguard
[39] G. Iadarola, F. Martinelli, F. Mercaldo, and A. Santone, "Towards an interpretable deep learning model for mobile malware detection and family identification," *Comput. Secur.*, vol. 105, 2021, Art. no. 102198.
[40] Y. Liu, C. Tantithamthavorn, L. Li, and Y. Liu, "Explainable AI for android malware detection: Towards understanding why the models perform so well?," in *Proc. IEEE 33rd Int. Symp. Softw. Rel. Eng.*, 2022, pp. 169–180.
[41] R. Pradhan, A. Lahiri, S. Galhotra, and B. Salimi, "Explainable AI: Foundations, applications, opportunities for data management research," in *Proc. Int. Conf. Manage. Data*, 2022, pp. 2452–2457.
[42] C. K. Tantithamthavorn and J. Jiarpakdee, "Explainable AI for software engineering," in *Proc. 36th IEEE/ACM Int. Conf. Automated Softw. Eng.*, 2021, pp. 1–2.
[43] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual explanations from deep networks via gradient-based localization," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2017, pp. 618–626.
[44] D. Han et al., "DeepAID: Interpreting and improving deep learning-based anomaly detection in security applications," in *Proc. ACM Conf. Comput. Commun. Secur.*, 2021, pp. 3197–3217.
[45] oleavr, "Frida," 2022. [Online]. Available: https://frida.re/
[46] "Droidmon," 2015. [Online]. Available: https://github.com/idanr1986/droidmon
[47] X. Sun, L. Li, T. F. Bissyandé, J. Klein, D. Octeau, and J. Grundy, "Taming reflection: An essential step toward whole-program analysis of android apps," *ACM Trans. Softw. Eng. Methodol.*, vol. 30, no. 3, pp. 1–36, 2021.
[48] A. Sadeghi, R. Jabbarvand, N. Ghorbani, H. Bagheri, and S. Malek, "A temporal permission analysis and enforcement framework for android," in *Proc. IEEE/ACM 40th Int. Conf. Softw. Eng.*, 2018, pp. 846–857.
[49] S. Wang et al., "APER: Evolution-aware runtime permission misuse detection for Android apps," in *Proc. 44th Int. Conf. Softw. Eng.*, 2022, pp. 125–137.
[50] "Maxim," 2022. [Online]. Available: https://github.com/zhangzhao4444/Maxim.git
[51] J. Ma, J. Li, W. Gao, Y. Yang, and K.-F. Wong, "Improving rumor detection by promoting information campaigns with transformer-based generative adversarial learning," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 3, pp. 2657–2670, Mar. 2023.
[52] H. You et al., "Castling-ViT: Compressing self-attention via switching towards linear-angular attention at vision transformer inference," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 14431–14442.
[53] A. Mohtashami and M. Jaggi, "Random-access infinite context length for transformers," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, pp. 1–13.
[54] D. Rau, M. Dehghani, and J. Kamps, "Revisiting bag of words document representations for efficient ranking with transformers," *ACM Trans. Inf. Syst.*, vol. 42, no. 5, pp. 1–27, 2024.
[55] L. Cui, J. Cui, Y. Ji, Z. Hao, L. Li, and Z. Ding, "API2Vec: Learning representations of API sequences for malware detection," in *Proc. 32nd ACM SIGSOFT Int. Symp. Softw. Testing Anal.*, 2023, pp. 261–273.
[56] H. G. Ramaswamy, "Ablation-cam: Visual explanations for deep convolutional network via gradient-free localization," in *Proc. IEEE/CVF Winter Conf. Appl. Comput. Vis.*, 2020, pp. 983–991.
[57] B. Zhou, Y. Sun, D. Bau, and A. Torralba, "Revisiting the importance of individual units in CNNs via ablation," 2018, *arXiv:1806.02891*.
[58] H.-X. Huang, J.-C. Li, and C.-L. Xiao, "A proposed iteration optimization approach integrating backpropagation neural network with genetic algorithm," *Expert Syst. Appl.*, vol. 42, no. 1, pp. 146–155, 2015.
[59] M. T. Ribeiro, S. Singh, and C. Guestrin, "Anchors: High-precision model-agnostic explanations," in *Proc. 32nd AAAI Conf. Artif. Intell.*, 2018, pp. 1527–1535.
[60] R. Guidotti, A. Monreale, S. Ruggieri, D. Pedreschi, F. Turini, and F. Giannotti, "Local rule-based explanations of black box decision systems," 2018, *arXiv:1805.10820*.
[61] W. Guo, D. Mu, J. Xu, P. Su, G. Wang, and X. Xing, "LEMNA: Explaining deep learning based security applications," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2018, pp. 364–379.
[62] Virustotal.com, 2023. [Online]. Available: https://www.virustotal.com/
[63] S. Mishra and D. Soni, "SMS phishing and mitigation approaches," in *Proc. 12th Int. Conf. Contemporary Comput.*, 2019, pp. 1–5.
[64] M. Liu, Y. Zhang, B. Liu, Z. Li, H. Duan, and D. Sun, "Detecting and characterizing SMS spearphishing attacks," in *Proc. Annu. Comput. Secur. Appl. Conf.*, 2021, pp. 930–943.
[65] Z. Lv, C. Peng, Z. Zhang, T. Su, K. Liu, and P. Yang, "Fastbot2: Reusable automated model-based GUI testing for android enhanced by reinforcement learning," in *Proc. 37th IEEE/ACM Int. Conf. Automated Softw. Eng.*, 2022, pp. 1–5.
[66] Fastbot, "Fastbot-android open source handbook," [Online]. Available: https://github.com/bytedance/Fastbot_Android?tab=readme-ov-file

Huijuan Zhu (Member, IEEE) received the master's degree from the School of Computer Science and Communication Engineering, Jiangsu University, Zhenjiang, China, in 2010, and the PhD degree from the School of Computer and Control Engineering, University of Chinese Academy of Sciences, Beijing, China, in 2017. Her research interests include software security, blockchain security, and deep learning. She is an associate professor with the School of Computer Science and Communication Engineering, Jiangsu University.

Xilong Chen is now studying with the School of Computer Science and Communication Engineering, Jiangsu University, Zhenjiang, China. His research interests include software security and deep learning.

Liangmin Wang (Member, IEEE) received the BS degree in computational mathematics from Jilin University, Changchun, China, in 1999, and the PhD degree in cryptology from Xidian University, Xi'an, China, in 2007. He is a full professor with the School of Cyber Science and Engineering, Southeast University, Nanjing, China. He has been honored as a "Wan-Jiang Scholar" of Anhui Province since Nov. 2013. Now his research interests include data security & privacy. He has published more than 70 technical papers at premium international journals and conferences, e.g., *IEEE/ACM Transactions on Networking* and IEEE International Conference on Computer Communications. He has severed as a TPC member of many IEEE conferences, such as IEEE ICC, IEEE HPCC, IEEE TrustCOM.

Authorized licensed use limited to: Jain University. Downloaded on January 05,2026 at 06:29:44 UTC from IEEE Xplore. Restrictions apply.

Zhicheng Xu is a student with the School of Mathematical Sciences, Jiangsu University, Zhenjiang, China. His research interests include malware detection and deep learning.

Victor S. Sheng (Senior Member, IEEE) received the master's degree in computer science from the University of New Brunswick, Canada, in 2003, and the PhD degree in computer science from Western University, Ontario, Canada, in 2007. He is an associate professor of computer science, Texas Tech University, and the founding director of the Data Analytics Lab (DAL). His research interests include data mining, machine learning, and related applications. He was an associate research scientist and NSERC postdoctoral fellow in information systems with Stern Business School, New York University, after he obtained his PhD. He is a lifetime member of ACM. He received the test-of-time award for research from KDD'20, the best paper award runner-up from KDD'08, and the best paper award from ICDM'11. He is an area chair and SPC/PC member for several international top conferences and an associate editor for several international journals.

Authorized licensed use limited to: Jain University. Downloaded on January 05,2026 at 06:29:44 UTC from IEEE Xplore. Restrictions apply.